

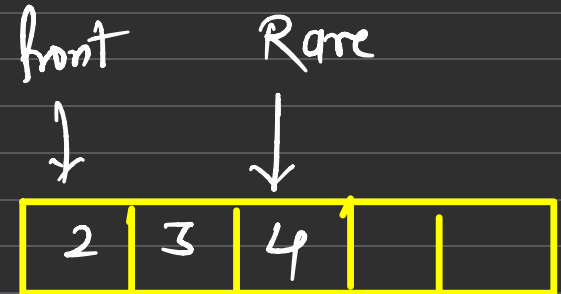
🎯 खुल जाएंगे सभी रास्ते, रुकावटों से लड़ तो सही..
सब होगा हासिल, तू जिद्द पर अड़ तो सही...

STACKS & QUEUES

IMPLEMENTATION
PATTERN TO ANSWER
PRACTICE PROBLEMS
APPLICATIONS

What is Queue

- Linear Data structure
- Insertion takes at one end & deletion at other end.
- Insertion $\rightarrow$ Rear End
- Deletion $\rightarrow$ front end
- FIFO order



Operations on Queue

enqueue: To insert element in queue

dequeue: To remove element in queue

Front: First element to delete.

isEmpty: To check if queue is empty

~~$\epsilon\phi(7)$~~ ~~$\epsilon\phi(1)$~~ ~~$\epsilon\phi(3)$~~ ~~$d\phi$~~ f ~~$d\phi$~~ ~~$\epsilon\phi(1)$~~
 ~~$d\phi$~~ $d\phi$

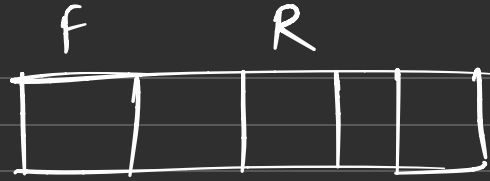
f						
R						
-1	0	1	2	3	4	5

Types of Queue

Linear Queue

Circular Queue

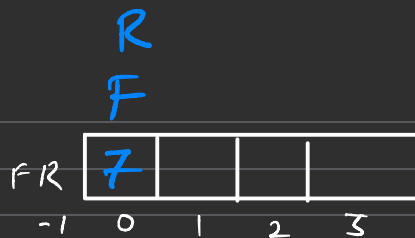
Deque — double Ended Queue



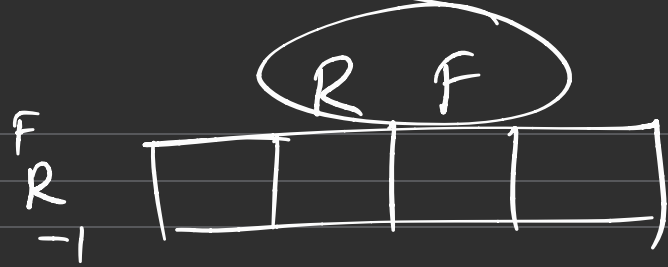
Implementing Linear Queue

```
void enqueue (int data)
{
    if (rear == N-1) return
    if (rear == -1)
    {
        rear++, front++
        q[rear] = data
    }
}
```

```
rear++
q[rear] = data
```



deQueue



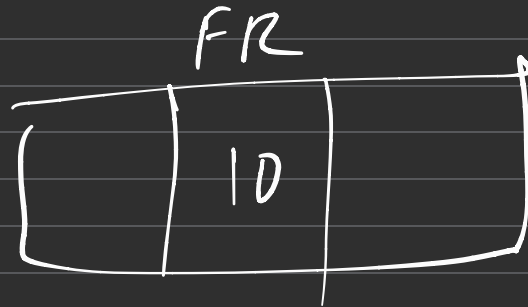
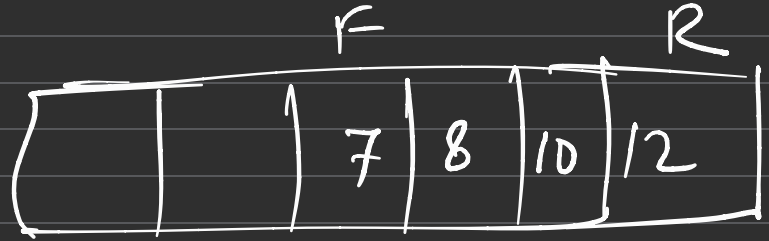
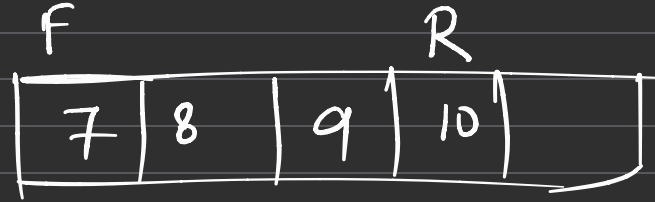
```
void deQueue()  
{  
    if (front > Rear) Underflow
```

```
    if (front == rear)  
    {  
        front = rear = -1  
    }  
    }
```

```
    F++
```

```
    }
```

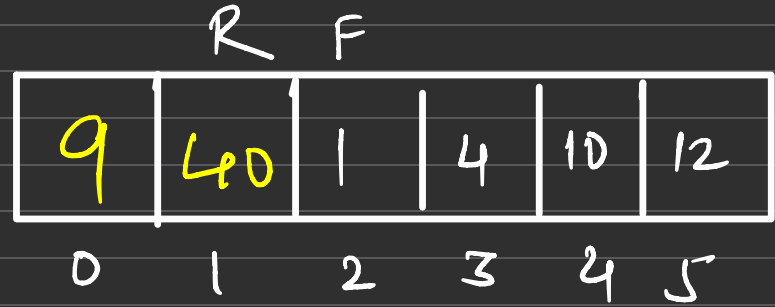
Traversal



Implementing Circular Queue

if (rear + 1 == front)

OF



N = 6

if (rear == -1)

{ front++, rear++

q[rear] = x

rear = (rear + 1) % N

q[rear] = data

```
void dequeue()
```

```
{  
    if (front == -1)
```

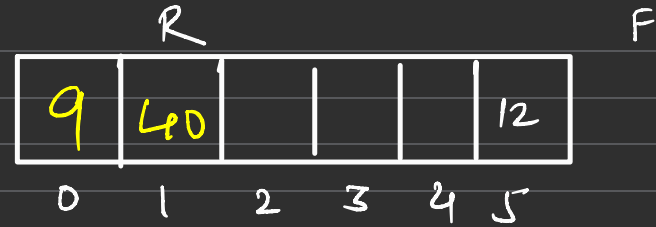
UF

```
    if (rear == front)
```

```
    {  
        front = rear = -1
```

```
    }
```

```
    F = (F + 1) % N
```



N=6

F = 7 7 4 8 6

R		F			
9	40	1	4	10	12
0	1	2	3	4	5

for (i = front : !rear : i++)

N=6

1 4 10 12

810MB

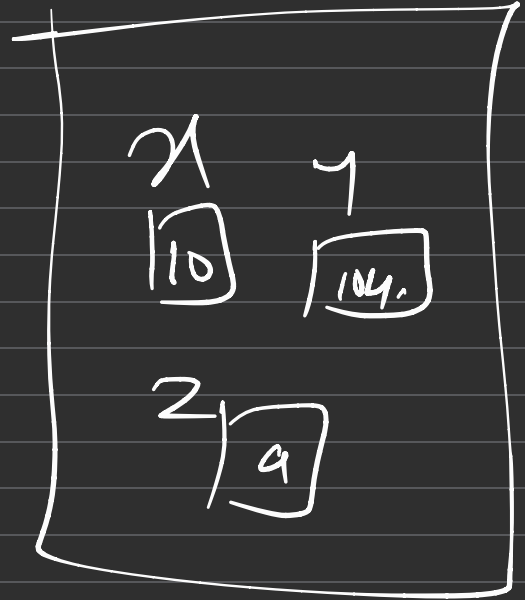
Buffer

8.8.7) Z

float y = 86.7 int x = 10

double y = 100.4

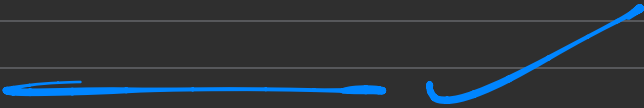
char Z = 'y' →



RAM

String

int



Sc. next Line

int char float
double
String
data type

Non-primitive

data type

```
char *p = "pramod"  
pf(p)
```

String

s = "pramod"

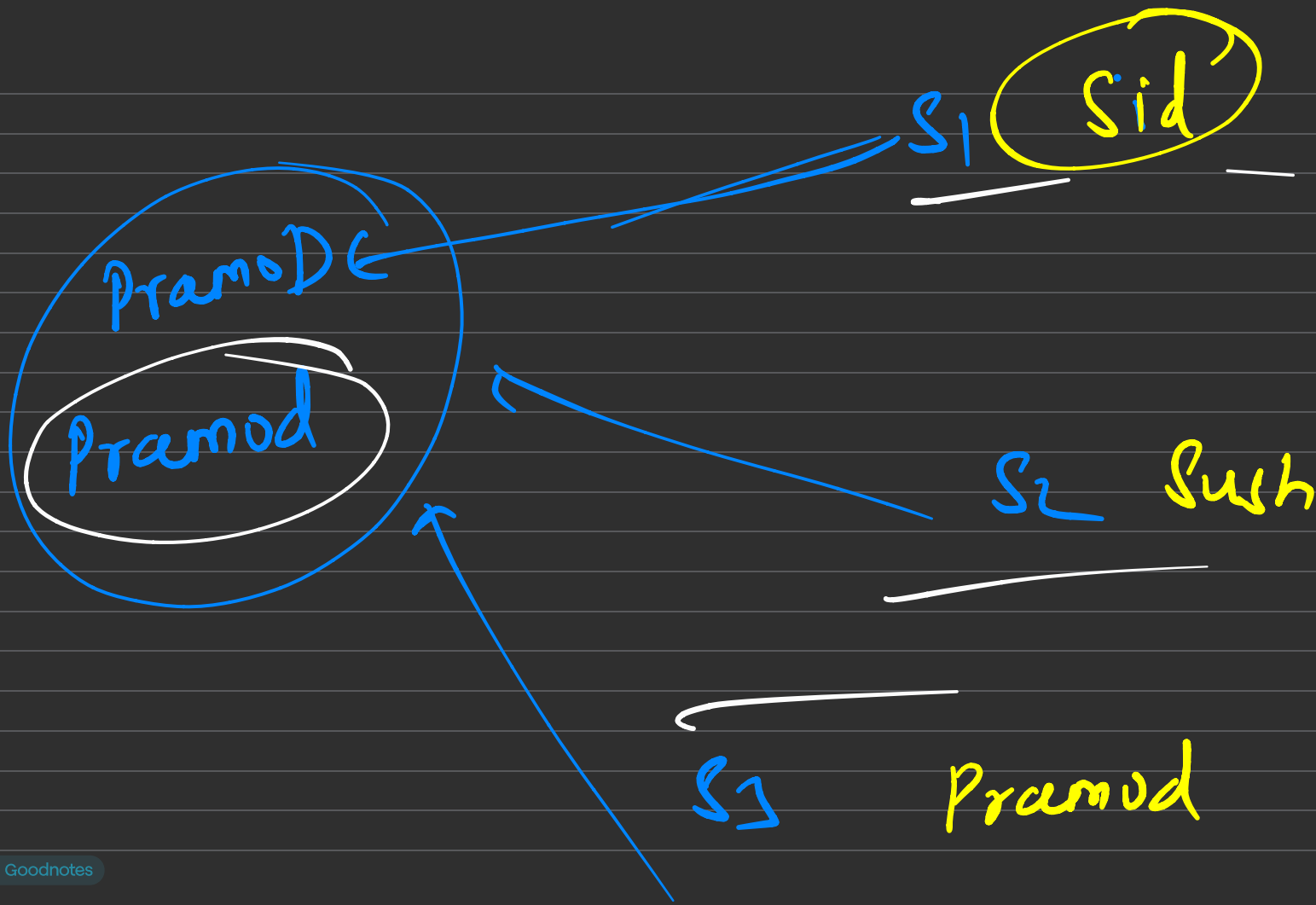
↓
Object

↓
Reference to
Object

↓
Class

Immutable

s₁ = s + "Bhosale"

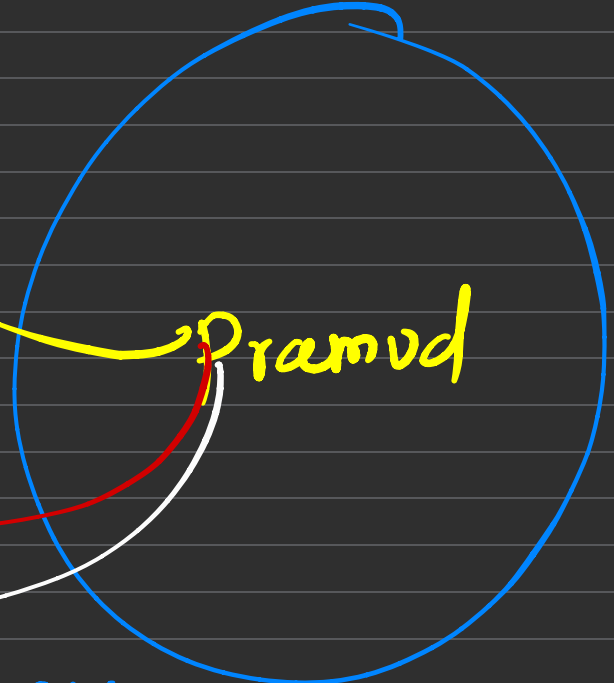


JUM

String s1 = "pramod"

String s2 = "pramod"

String p = "pramod"



String constant pool

